

The need for the integration of ideas from the three research areas “User Modeling”, “Ubiquitous Computing” and “Semantic Web” is inherent in the definition of the basic concept of `SITUATIONALSTATEMENTS`, that is introduced in this chapter. A situational statement carries information about user model entries, context data or low-level sensor data. It forms the central, uniform structure for representing any situational information. The first part of this chapter presents a new solution to the issue of decomposing descriptions of complex situations into structured, uniform sentences, so-called `SITUATIONALSTATEMENTS`. In the second part of this chapter, the concept of `SITUATIONALSTATEMENTS` is realized by the XML application `UserML`, which serves as an exchange language for ubiquitous user-adaptive systems.

4.1 How to Represent Information about Complex Situations?

The question that arises from the point of view of knowledge representation is: *How can all these different types of contextual information be represented in a handy, uniform way?* On the way to find a solution, the story behind the picture in figure 4.1 is extended, analyzed and annotated with semantical meta information.

4.1.1 Situational Descriptions in the Airport Scenario

Let us assume that an inference system deduces that *the passenger Peter is now most probably under high time pressure because he is near the duty-free shop of the airport, while boarding of his flight closes in a few minutes. Additionally, his walking speed sensors report fast walking. According to his privacy settings, this information is only freely available to preselected people and systems.*

The same description could be annotated as: “An inference system (`creator`) deduces that Peter (`subject`) is now (`start`) most probably (`confidence`) under high (`object`)¹ time pressure (`predicate`), because (`evidence`) he is near the duty-free

¹The naming of the `object` and `predicate` is inherited from the naming of the original RDF triple, while



Figure 4.1: This image depicts a typical complex situation of a passenger at an airport. He is interacting with a mobile device, probably a pedestrian navigation system, carrying some luggage over his shoulder while hurrying towards a gate. This image produces the impression of time pressure and slight despair. The bridge between such an extreme situation and human-computer interaction research is the fundamental need for adaptation in the communication between this passenger and his instrumented environment.

shop (position) of the airport (location), while boarding of his flight closes in a few minutes (evidence). Additionally, his walking speed sensors (`creator2`) report fast walking (`predicate2`, `object2`). According to his (owner) privacy settings (`privacy`), this information is only freely available to preselected people and systems (`access`).”

The first insight from this annotation example is that one can separate the whole description of this airport scenario into smaller, sentence-like units, that is to say into a sentence about Peter’s time pressure, a sentence about Peter’s walking speed, and probably a sentence about the boarding time of Peter’s flight. However, these intuitive semantic roles do not lead straight forward to a clear structure, if one wants to omit the complexity of natural language.

4.1.2 Remarks on Design Decisions for the User Model Language

The following three different approaches from knowledge representation research influenced the design decisions for the model of SITUATIONALSTATEMENTS, which is defined in the next section:

- the resource description framework RDF, with subject-predicate-object triples, reification, collections and all the theory of semantic web in support, as well as other semantic web languages like OIL, DAML or OWL, see chapter 2.3 for an introduction.

the first one carries the *value* and the second one the *attribute*.

- the FrameNet approach, see section 3.3.4 for a short introduction, with the annotations playing the role of frame elements, allows the links between semantics and syntax in the results of lexical analysis to be formalized. Semantic frames are schematic representations of situations involving various participants, props, and other conceptual roles.
- extended markup language applications concerning users or humans in general, like HumanML², the Human Markup Language, VHML³, the Virtual Human Markup Language, EML⁴, the Emotional Markup Language or CPExchange⁵, the Customer Profile Exchange

The analysis of related markup languages and approaches led to an early decision to create a new, specialized user modeling (markup) language, with the acronym UserML⁶. One purpose of the UserML project is to develop a platform for communication about partial user models in a ubiquitous computing environment, where all different kinds of systems work together to satisfy the user's needs. Standardization is of great importance but individual applications also call for very specific solutions.

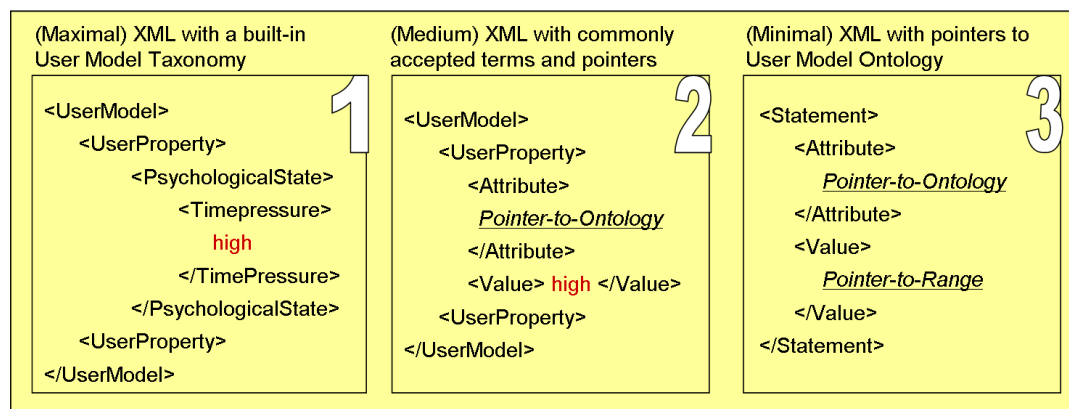


Figure 4.2: Three design drafts for a user model representation language, the first one with the built-in user model taxonomy, the second one with a commonly accepted, minimal taxonomy and the third one only with pointers to a user model ontology

Three different ways of forming user model information into the XML tree structure are illustrated in figure 4.2. All three have advantages and disadvantages. The first approach transforms the whole complexity of the domain knowledge into the tree structure. The advantage is that no external taxonomy or ontology is needed, but the main disadvantage is the acceptance in the research community of any fixed definition and the maintainability for

²HumanML homepage: <http://www.humanmarkup.org/>

³Virtual Human Markup Language homepage: <http://www.vhml.org/>

⁴Emotional Markup Language is part of the VHML project:

<http://www.interface.computing.edu.au/documents/VHML/2001/>

⁵Customer Profile Exchange homepage: <http://www.cpexchange.org/>

⁶see [Heckmann, 2002a] for a discussion

extensions if the user model dimensions are coded into the DTD or XML-Schema definition. Another important disadvantage is which point of view should be chosen and with which focus? Some possibilities for instance:

- **the user seen as human-being:** human-being related properties like emotions, cognition, or physical descriptions could form the main focus
- **the use of user-adaptive systems:** putting the user-adaptive systems in the main focus causes the problem that demand-orientation and system-dependency could lead to too system-specific structures
- **user interaction centered:** the user interacts with his/her environment including computing systems, context and the whole situation, resulting in the problem of modeling the environment with the whole interaction process

The idea for the second approach in figure 4.2 of combining the minimal accepted common user model taxonomy with pointers to external ontologies was developed in a discussion with Henry Thompson⁷, see [Carolis et al., 2003] for a similar approach. Surprisingly, not even a minimal intersection of top-level terms could be found in the user modeling research community⁸. For instance the concept of a *usage model* is widely used, see [Kobsa, 2001a], while on the other hand, the concept of a *personal journal*, see [Kleinbauer et al., 2003], has also been developed. The attempt to find a common, merged taxonomy for the three research areas of user modeling, context-awareness and resource-adaptivity seemed impossible.

The third approach in figure 4.2, however, showed a solution to the problems of the previous two. Thus, the best solution to the information design task seemed to be the complete separation of the domain of user modeling (as well as the domain of context-awareness, see figure 4.3 for a simplified taxonomy for the airport scenario), and the structure of the representation language. The user model domain knowledge will be modeled in a user model ontology and the structure in an extended resource-description-like framework.

The advantage of this approach is that if the research community subgroups can not find a commonly accepted user model ontology, they can all use their own ones, but still use the same framework. Of course, the third design draft of figure 4.2 is not powerful enough to cover all the demands of modern user modeling tasks, since it only offers the simple structure of attribute-value pairs. However, it points towards the resource description framework and further to an extension of the resource description framework, which is described in the following section. Another important advantage of this approach is that the property information about the object of interest is equally represented in user-adaptive systems, context-awareness and resource-adaptive computing, as described in [Heckmann, 2003b].

⁷Discussion with Henry Thompson on the design for a user modeling markup language at the University of Edinburgh, 2002

⁸User Modeling Research Community homepage: <http://www.um.org>

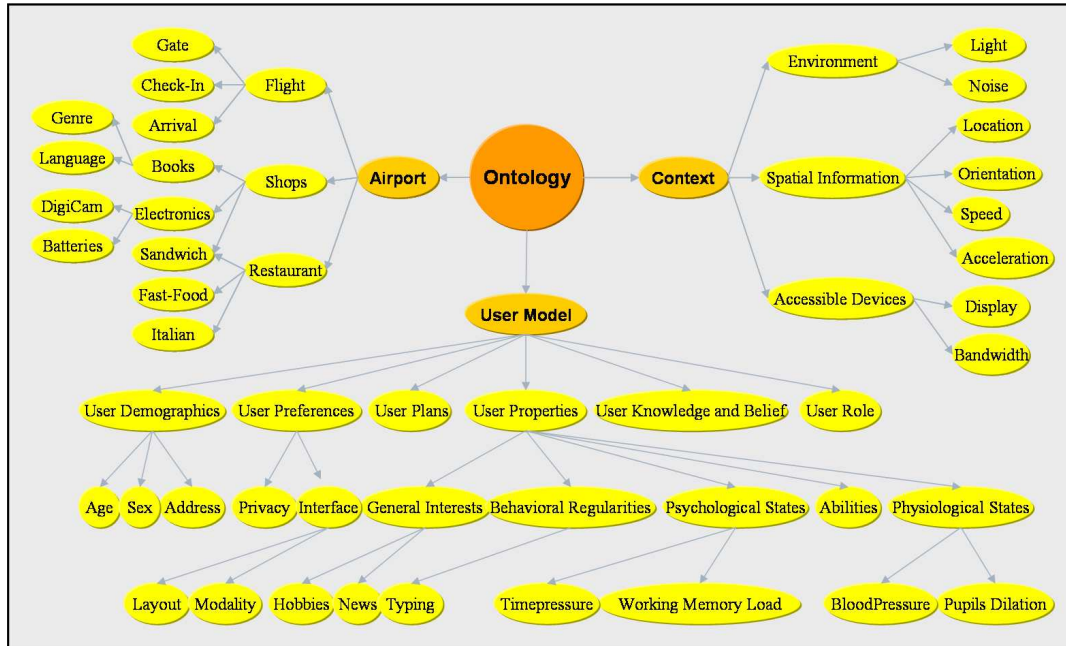


Figure 4.3: A simplified, partial taxonomy for the airport scenario, which is divided into three sub parts, namely the airport related domain concepts on the left, the context related concepts on the right, and the user model related concepts on the bottom

4.1.3 From RDF to Extended Triples

Together with the reification mechanism, the resource description framework RDF⁹ forms a powerful tool for knowledge representation. RDF is based on triples of subject, predicate and object as shown in figure 4.4: a resource, the subject, is linked to another resource, the object, denoting the value through an arc labeled with a third resource, the predicate. It is said that the subject has a property predicate, valued by object.

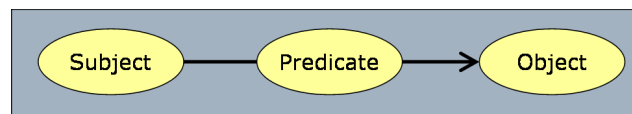


Figure 4.4: The basic RDF triple with subject, predicate and object

With this basic mechanism, one could already represent user model information like: “the subject=*Peter* has the predicate=*CognitiveLoad* with the object=*high*”. However, if one needs to add more information to this description, for example a temporal aspect like “the time when *Peter* was under high cognitive load”, RDF offers either the

⁹Resource Description Framework homepage: <http://www.w3c.org/rdf>

reification mechanism with which the basic triple is transformed into one new resource, that can be used as a whole in other statements, or the construct of collections. Setting up a specialized structure within RDF was introduced in [Heckmann, 2002a] as a third possibility into the concept of *Extended Triples*. See figure 4.5 for a diagrammatic view to this extended-triple RDF-application.

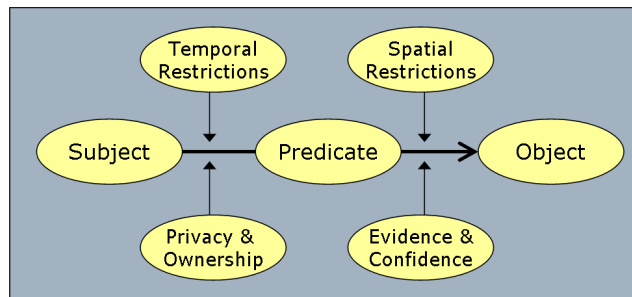


Figure 4.5: The extended RDF triple with subject, predicate and object, as well as temporal restrictions, spatial restrictions, privacy and ownership, evidence and confidence

RDF is extended with predefined semantical slots, like *owner* or *confidence* instead of handling triples with reification. The meta level information of a statement like “*who is responsible for this piece of information?*”, or “*what is the confidence value for it?*”, is combined with the actual content of the statement like “*the cognitive load of this person is high*”. Furthermore the basic information content is enriched with temporal and spatial constraints like “*this property is valid between now and tomorrow*”.

The so-called *Extended Triple* is in fact a 7-tuple, with temporal- and spatial restrictions as well as meta-data about ownership, privacy, evidence and confidence. The idea is to allow more powerful, but still structured, statements about situations. The new arrow type in figure 4.5 has the meaning of *adding information* to the basic triples. Nevertheless, the graph of the extended triples did not prove to be flexible enough for further extensions and more importantly, from an implementation point of view, did not prove to be easily transformable into the relational database system. Therefore, the idea of extending basic RDF finally led to the structure of SITUATIONALSTATEMENTS, which is defined in the following section.

4.2 The Model of SITUATIONALSTATEMENTS

SITUATIONALSTATEMENTS represent partial descriptions of situations like user model entries, context information or low-level sensor data. They form the main data structure in the given situation concept. What is new and special about SITUATIONALSTATEMENTS is the extensive layered approach shown in the onion model of figure 4.6. The information is organized in a predefined hierarchy of meta levels wrapped around the mainpart information.

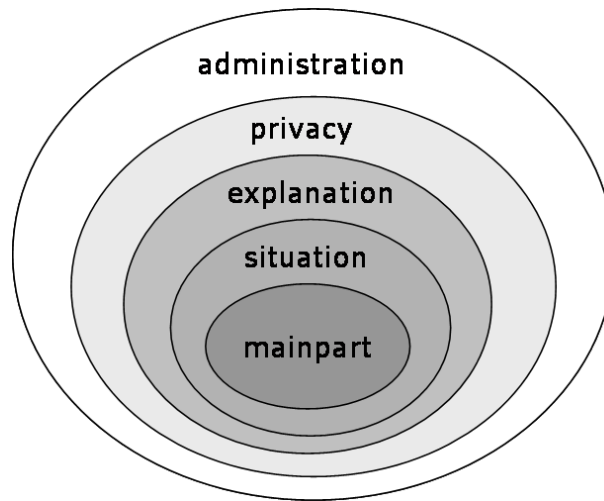
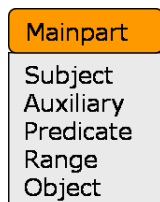


Figure 4.6: *Onion model to represent the five layers of situational statements*

These layers of meta level information within SITUATIONALSTATEMENTS can be seen as a collection of slots or attributes that are arranged in five boxes, namely the mainpart box, the situation box, the explanation box, the privacy box and the administration box. These boxes have an organizing and structuring functionality. Their meaning is explained in detail in the following subsections.

4.2.1 The Mainpart Box



The leading content information of the described partial situation can be expressed by the attribute-value pairs of the mainpart box. The three attributes `subject`, `predicate` and `object` are necessary attributes for each SITUATIONALSTATEMENT, while `auxiliary` and `range` are optional attributes. The mainpart attributes are directly derived from the theory of RDF triples with subject-predicate-object and form a superset of them, see section 4.4 for a discussion. That means that every RDF triple can be expressed by a SITUATIONALSTATEMENT. On the other

hand, all SITUATIONALSTATEMENTS can be translated back into RDF documents with reification, see section 4.4.3 for a short introduction. The different syntactic specifications are shown in the appendix section A.1.

A basic example of information that could be found in a simple user model is: `subject="Peter"`; `predicate="time pressure"`; `object="high"`; with the intended meaning that *Peter has high time pressure*.

A second example that could be expressed by a context-aware system is: `subject="mobile phone"`; `predicate="battery power"`; `object="low"`; with the intended meaning that *the actual battery power of the mobile phone is low*.

The original RDF predicate is split into `auxiliary`, `predicate` and `range` in order to be more flexible in expressing properties. A new surplus in expressivity according to RDF is the auxiliary construction. It was introduced as a unifying element with respect to the special needs of knowledge representation with pointers to external ontologies. To demonstrate the necessity of this auxiliary construction the following example is given: the term *teaching* is defined and described in a semantic web ontology, while a user-adaptive system might want to represent the three statements:

- *Peter is currently teaching*
- *Peter likes teaching very much*
- *Peter knows a lot about teaching*

With the means of `SITUATIONALSTATEMENTS`, the user-adaptive system would define `subject="Peter"`, `predicate="teaching"` and the corresponding `auxiliary` as *"interest"* or *"knowledge"*. A predefined list of auxiliaries can be found in section 5.2.

The `range` attribute offers a new degree of freedom to the ontology definition: it decouples the definition for the predicate from possible range scales. The following examples illustrate this argument:

- *Peter's time pressure is low (within a scale of low-medium-high)*
- *Peter's time pressure is 0.6 (within a numeric scale between 0 and 2)*
- *Peter's time pressure is 30% (within 0% - 100%)*

In terms of `SITUATIONALSTATEMENTS`, one would define `subject="Peter"`, `predicate="time pressure"` and `range="low-medium-high"` or `range="numeric-0-2"`, or `range="percentage"` respectively, while these ranges are defined in a special part of the ontology. A mapping between different ranges, which is also defined in the ontology, is especially interesting for conflict resolution in chapter 6 and user model integration in chapter 7.

A more elaborate approach with uncertain knowledge, probability distributions and reasoning under uncertainty could also be integrated in `SITUATIONALSTATEMENTS`. For example, the *Dempster-Shafer Theory*, see [Dempster, 1968] and [Shafer, 1979] with measures of belief, based on evidences, or Bayesian probability distributions, see for example [Jensen, 2001], could be interesting for later extensions. However, at this point the focus is set on representational issues that stem from RDF. The *Discourse Representation Theory*, see [Kamp and Reyle, 1990], is used in natural language processing, to dynamically interpret so called *discourses* (linguistic units composed of several sentences like conversations, arguments or speeches). This theory provides a logical framework that could have been interesting to model `SITUATIONALSTATEMENTS`. However, since no connection is given towards the Semantic Web, this theory was not further recognized.

To summarize, the mainpart box extends the RDF triple with the attributes `auxiliary` and `range` in order to simplify and strengthen the definition of the general user model ontology GUMO.

4.2.2 The Situation Box

Situation
Start
End
Durability
Location
Position

Information in the situation box is responsible for the temporal and spatial embedding of the whole statement in the real physical world. Since this situational aspect is especially important and dominant in ubiquitous computing, it gave its name to the whole concept of SITUATIONALSTATEMENTS. All attributes in this box are optional. The attribute `start` denotes the point in time when the statement becomes valid. The default value is the date-time when this statement was given. The attribute `end` denotes the point in time when the validity of the statement ends. This ending could be in the past, in the present or in the future. With this open approach one can handle the issue of the history in user modeling and context-awareness. Earlier statements are still available, they are not overridden or replaced by newer values. Instead the query answer mechanism returns - by default - the current value by temporal inference. Furthermore, the attribute `durability` carries the qualitative time span of how long the statement is expected to be valid (minutes, hours, days, years). In most cases when user model dimensions or context dimensions are measured, one has a rough idea about the expected durability, for instance, emotional states change normally within hours, however personality traits won't change within months. Since this qualitative time span is dependent on every user model dimension, a definition mechanism is prepared within the general user model ontology GUMO. Some examples of rough `durability` classifications are:

- `physiologicalState.heartbeat` - can change within seconds
- `mentalState.timePressure` - can change within minutes
- `emotionalState.happiness` - can change within a few minutes
- `characteristics.inventive` - can change within months
- `personality.introvert` - can change within years
- `demographics.birthplace` - can't change at all (under normal circumstances)

The idea is that if no new actual value is available after a while, one can still work with old values combined with probably reduced confidence values. The suggested terms for durability are not hard coded into the SITUATIONALSTATEMENT approach, but a predefined setting for possible values can be found in appendix B.2.3 on page 236. In addition to the duration time interval, together with a given requesting time, the complex temporal inference process can calculate the status of the statement like *not expired*, *just expired* or *expired*, see [Schmidt, 2005] for a discussion on controlled aging of user context.

The attribute `location` defines the qualitative spatial location, where this statement takes place, in relation to the specialized spatial model for ubiquitous computing that is presented in the UBISWORLD ONTOLOGY section 5.3.2 from page 105 onwards, while the attribute `position` allows to specify the exact quantitative coordinates.

4.2.3 The Explanation Box

Explanation

Source
Creator
Method
Evidence
Confidence

The collection of explanatory meta attributes is partly required by the user's right for explanation and partly by the conflict resolution mechanism for inferred user modeling data, see chapter 6.3. The `source` attribute contains the statement's origin, that is to say where the statement was stored, or from which sensor measurement it came from. With the help of this attribute, double entries can be detected in the user model integration process, see chapter 7.1. The user's right to ask for explanation is handled with the next three attributes: the `creator` names the system or person that is responsible for the creation of this statement. The `method` attribute leads to the answer "how" the mainpart was inferred, deduced or measured. Its attribute's value can either be a pointer to an ontology or free text. The list of evidence that supports the statement can be stored in the `evidence` attribute, while the `confidence` provides a place to store the creator's expected level of confidence in the statement. All attributes in the explanation box are optional.

4.2.4 The Privacy Box

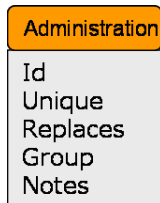
Privacy

Key
Owner
Access
Purpose
Retention

The privacy meta attributes are very important for the acceptance of user modeling data, since these attributes allow controlled distribution of sensitive data. The `key` attribute which forms an encrypted security key can be attached to the statement and lock it if the key can not be resolved. The `owner` attribute refers to the person or system that is in charge of managing the distribution and editing of the three following attributes: `access`, `purpose`, `retention`. The first one can contain the class of users or systems that are allowed to read the statement. The second one can put restrictions on the intention for which the statement may be used. And finally the third one controls how long the statement may be kept or used. See chapter 8.4.2 for a detailed description of privacy handling. The possible values for the `access` attribute are *public*, *friends* and *private*, for the `purpose` attribute: *commercial*, *research* and *minimal* and for the `retention` attribute: *long*, *middle* and *short*. According to these settings, the privacy handler will grant or deny access and delete the appropriate statements for privacy reasons.

New in this privacy approach is that the owner's intended privacy settings go with the statement itself on the information journey when it is exchanged into a distributed ubiquitous network. In other current user model services, once the information is given out to some client, the privacy information is not attached any more, and this client could spread this statement under new privacy settings.

4.2.5 The Administration Box



The administration box is the outermost layer in the layered onion model. It is meant to fasten organizational issues in huge sets of situational statements. The `id` attribute contains a locally unique identifier for referencing the statement and is mostly used as a primary key in a database. The `unique` attribute however forms a globally unique identifier for referencing the statement throughout the whole internet. It could either be a URI¹⁰ or a GUID¹¹. Since in our approach we cannot guarantee that the creator of the statement has rights or the means to delete a statement

after creation, the `replaces` attribute makes it possible to state which statement should be replaced by a newer one. Especially sensors that constantly emit more up-to-date data receive the identification numbers of their last situational statements from the user model server as a return value and therefore replace older information without deleting it. Figure 4.7 on page 63 shows a rough classification of the statements into groups like *UserModel*,

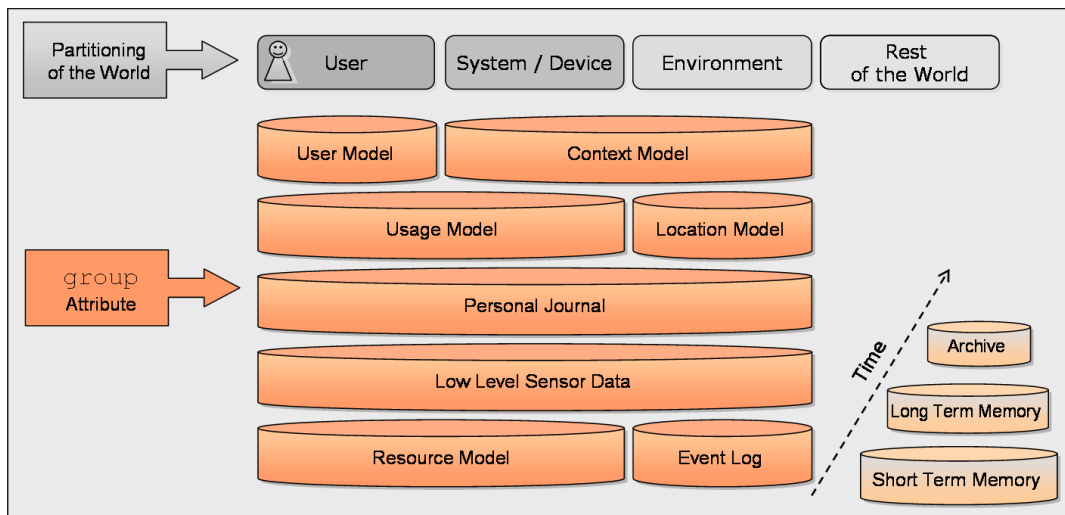


Figure 4.7: Possible values for the *group* attribute (within the administration box) in relation to the conceptual partition of the world and a temporal dimension

UsageModel, *ContextModel* or *LowLevelSensorData* that can be expressed by the *group* attribute. However, this grouping is only of minor importance in our approach, since the statements are conceptually and physically stored at the same place. It is used within the smart situation retrieval process for efficiency reasons. In related systems, for example in the SPECTER system¹², a clear definition of these memory models plays a more important role, since each group is stored and treated independently, see e.g. [Kröner, 2005]. And finally,

¹⁰The W3C URL/URI homepage: <http://www.w3.org/Addressing/>

¹¹A web service for the generation of *Globally Unique Identifiers* can for example be found at: <http://www.hoskinson.net/WebServices/GuidGenerator.aspx>

¹²SPECTER homepage: <http://www.dfki.de/specter>

the `notes` attribute has no predefined semantics and can contain additional unstructured meta information about the statement.

To summarize, the information represented in a `SITUATIONALSTATEMENT` is arranged in a layered fashion of five times five attributes. The mainpart attribute group contains the actual content of the statement, the situation group defines necessary temporal and spatial side conditions, the explanation group covers the creator and the creation-method as well as the evidence and confidence. The privacy group offers the possibility of expressing privacy meta data, while the attributes of the administration group cover the identification, replacement and grouping of the statements. A complete listing of these attributes, together with short descriptions, can be found in table 4.1.

Definition 4.1 (Situational Statement) A `SITUATIONALSTATEMENT` is a collection of attributes as defined in table 4.1. The term `STATEMENT` without the prefix serves as an abbreviation.

4.3 The Model of `SITUATIONREPORTS`

Individual `SITUATIONALSTATEMENTS` are only able to carry the information of one single entity or - more accurately - of one statement. The question of arranging several statements into a new data structure is now interesting.

4.3.1 From Situational Descriptions to `SITUATIONREPORTS`

The informal situational descriptions, as presented in section 4.1.1, can now be formalized with the help of `SITUATIONALSTATEMENTS` and an additional data structure. The easiest data structure for collecting several statements into one unit is the *bag*, which is a *set* with possible multiple elements. Figure 4.8 depicts a general `SITUATIONREPORT` schema, which is defined as an unordered collection of `SITUATIONALSTATEMENTS`.

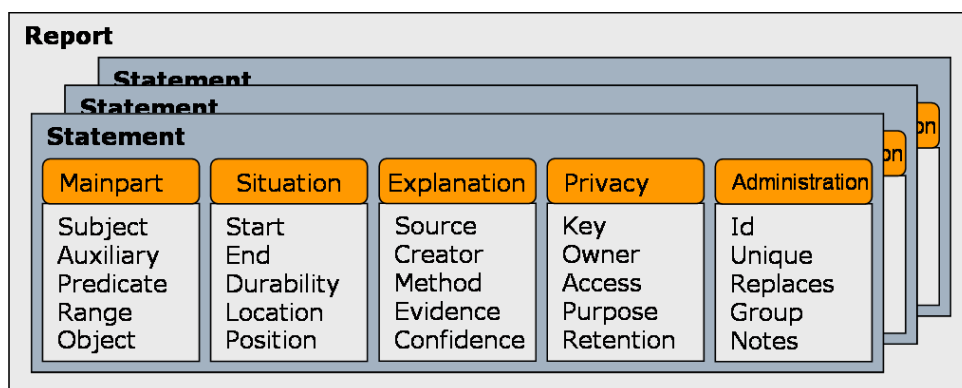


Figure 4.8: A `SITUATIONREPORT` is defined as a bag of `SITUATIONALSTATEMENTS`

Mainpart	<i>the basic, rdf-based five-tuple of situational statement attributes</i>
subject	the main entity the statement is about (mostly the user)
auxiliary	the auxiliary part of the predicate like “hasProperty” or “hasInterest”
predicate	the dimension or category of the user like “Age” or “Swimming”
range	the range of the object attribute, defaults are possible
object	the value for the subject-auxiliary-predicate triple of the statement

Situation	<i>temporal and spatial constraints, restricting the mainpart</i>
start	the point of time when this statement was given
end	the point of time when this statement is no longer valid
durability	the qualitative time span of how long the statement is valid
location	the qualitative spatial location where this statement takes place
position	the quantitative spatial location, the exact coordinates

Explanation	<i>a collection of explanatory meta attributes</i>
source	the origin where the statement is stored
creator	the person or system that is responsible for the creation of this statement
method	the manner by which the statement was created
evidence	a list of evidence that supports the statement
confidence	a number that displays the creator’s expected truth of the statement

Privacy	<i>a collection of privacy meta attributes, controlling the distribution</i>
key	optional encrypted security key that can be attached to the statement
owner	the person or system that may manage the distribution
access	the class of users or systems that are allowed to use the statement
purpose	the qualitative purpose for which the statement may be used
retention	the qualitative time of how long the statement may be kept or used

Administration	<i>a collection of administrative meta attributes</i>
id	a locally unique identifier for referencing the statement in the database
unique	a globally unique identifier for referencing the statement
replaces	a unique identifier of another statement that has to be replaced
group	rough classification of the statement like “UserModel” or “SensorData”
notes	an additional attribute with free semantics, can serve as a variable

Table 4.1: The intended meaning of the twenty-five predefined attributes of Situational-Statements, containing main content information, as well as situational, explanatory, access and administrative meta-information, in order to describe complex situations. This selection forms a minimal set for the special needs according to the integration of user modeling, ubiquitous computing and semantic web.

Definition 4.2 (Situation Report) A SITUATIONREPORT is an unordered collection of SITUATIONALSTATEMENTS, that belong to the same situational description. The term REPORT is used as an abbreviation.

4.3.2 A SITUATIONREPORT in the Airport Scenario

The partial situation model of the airport scenario of section 4.1.1 on page 53 could be represented by the three SITUATIONALSTATEMENTS of figure 4.9. However, the attribute values are not correctly presented. They are only indicated and should be understood as simplified resources. The definition of these values and their datatypes can partly be found in chapter 5.

Situational Statement / Box	Situational Statement / Box	Situational Statement / Box
Mainpart	Mainpart	Mainpart
Subject = Peter Auxiliary = hasProperty Predicate = timePressure Range = low-medium-high Object = high	Subject = Peter Auxiliary = hasProperty Predicate = walkingSpeed Range = slow-medium-fast Object = fast	Subject = Flight LH225 Auxiliary = hasPlan Predicate = boarding Range = time Object = in 10 minutes
Situation	Situation	Situation
Start = 2003-04-16T19:15 End = ? Durability = few minutes Location = airport.dutyfree Position = x34-y22-z15	Start = 2003-04-16T19:14 End = ? Durability = few minutes Location = airport.dutyfree Position = x32.y23.z15	Start = 2003-04-16T19:14 End = ? Durability = few minutes Location = airport.gate23 Position = ?
Explanation	Explanation	Explanation
Source = peter.repository Creator = airport.inference Method = deduction13 Evidence = id2, id3 Confidence = most-probably	Source = sensor.repository Creator = sensor.PW Method = Bayes Evidence = LowLevelData Confidence = 0.8	Source = airport.repository Creator = airport.inference Method = deduction13 Evidence = fight-system Confidence = 0.6
Privacy	Privacy	Privacy
Key = ? Owner = Peter Access = friends-only Purpose = research Retention = 1 day	Key = ? Owner = Peter Access = friends-only Purpose = research Retention = 1 week	Key = ***** Owner = Airport Access = public Purpose = commercial Retention = 1 month
Administration	Administration	Administration
id = 1 unique = u2m.org#154123 replaces = ? group = UserModel notes = ;-(	id = 2 unique = u2m.org#154124 replaces = u2m.org#154109 group = UserModel notes = ;-	id = 3 unique = u2m.org#154125 replaces = u2m.org#152148 group = ContextModel notes = ;-

Figure 4.9: SITUATIONREPORT with three SITUATIONALSTATEMENTS from the airport scenario, with meta data plus the three mainparts: (1) Peter is most probably under high time pressure, (2) Peter is walking fast, and (3) Boarding of flight LH225 will close in 10 minutes

4.3.3 Aggregating Statements in SITUATIONALREPOSITORIES

“Belonging to the same situational description” distinguishes the SITUATIONREPORTS from the most general collection of statements, namely the SITUATIONALREPOSITORY, that are meant to save statements that do not necessarily belong to the same situation. SITUATIONAL-REPOSITORIES serve as containers for any kind of SITUATIONALSTATEMENTS. An important additional property is that repositories must be identifiable in order to state the origin of a statement after it has been exchanged.

Definition 4.3 (Situational Repository) A SITUATIONALREPOSITORY is a uniquely identified collection of SITUATIONALSTATEMENTS. The term REPOSITORY is used as abbreviation.

However, the intended semantics for the attributes of SITUATIONALSTATEMENTS, together with their under-specified data types, leave some scope for variations within concrete instantiations of this conceptual model in user-adaptive services. Such a concrete, further specified instance of this rather general model of SITUATIONREPORTS has been defined and implemented along with SituationML in section 4.4.

4.3.4 Remarks about the Closed World Assumption

Intelligent behavior depends on the knowledge that is known about the environment. Much of this knowledge is descriptive and can be expressed for example in declarative situational statements. For the semantic interpretation of statements it is important to know the standpoint, regarding the frame axiom and the closed word assumption.

A theory (or knowledge base) is called *complete* if either every ground atom (here, the auxiliary-predicate-range triple) or its negation is in the theory. If the statement “*It is raining at location X*” is true, the negation of this sentence “*It is not raining at location X*” must be false. The question “*Is it raining at location Y?*” while no weather information is given for location Y, leads to the issue of incomplete knowledge and the question whether one should assume that it is not raining there because it was not explicitly stated. The *closed world assumption* is a straight forward convention to complete a theory in the way that the negation of a statement is assumed if the statement cannot logically be deduced from the theory.

For the work with SITUATIONREPORTS we do not apply the closed world assumption, since we allow contradicting statements at the same time, that could be resolved by conflict resolution as shown in section 6.3. We do not intend to model a complete blocks world, but we intend to model parts of the real world, see section 5.3. Or more precise, we intend to model situations in the sense of [Barwise and Perry, 1983], that correspond to the limited parts of reality we perceive, reason about and live in.

4.4 The Syntax of SituationML and UserML

The overall idea of integrating the three research areas of “User Modeling”, “Ubiquitous Computing” and “Semantic Web” is especially inherent in the definitions in this section.

The general model of SITUATIONALSTATEMENTS, as proposed in the previous section, is transformed into several XML applications, as well as RDF applications and OWL applications. These syntactical variations are defined in parallel, since special circumstances need specialized tools, i.e. the same information needs to be read by human beings, it needs to be prepared for inference systems and integrated into semantic web ontologies, it needs to fulfil database constraints for storage, and at the same time the transfer volume has to be optimized for mobile interaction settings. Because of these manifold demands, and since the SITUATIONALSTATEMENTS form the fundamental background of the whole u2m.org-research, an investigation of the information design possibilities has been undertaken.

The XML application family *SituationML* includes a concrete realization of the abstract model of SITUATIONREPORTS and SITUATIONALSTATEMENTS. *SituationML* can be read as *Situation Markup Language*. However, since SITUATIONALSTATEMENTS also cover the conceptual purpose of user models and context models, the terms *UserML* and *ContextML* are also used as abbreviations. The acronym *UserML* stands simultaneously for *User Model Markup Language*, as well as *User Model Exchange Language*. Analogically, the acronym *ContextML* stands for *Context Markup Language*, as well as *Context Exchange Language*.

4.4.1 SITUATIONALSTATEMENTS within SituationML

The following list in figure 4.10 presents the default representation for SITUATIONALSTATEMENTS within *SituationML*. All information is put into XML-elements, while the corresponding attribute groups like *explanation* or *privacy* are omitted for simplification reasons, since a flat tree structure is the easiest for prototyping. No attributes are used. Empty elements can be omitted, which leads to short statements, if no meta data are needed. The root element is named `statement`.

The variables from `a1` to `a25` do not necessarily carry the information in simple structures. Complex data types and ranges are defined in section 4.5. At this point, the focus lies on the pure design decision of the top level XML structure.

The following section presents three alternative syntax variations that all have their own advantages and disadvantages. The document type descriptions (the DTDs) of *SituationML* specifications are listed in appendix A.2, the XML Schema is listed together with its schema tree in the appendix A.3.

4.4.2 Syntax Variations on SituationML/XML

In the first syntax variation on the left hand side of figure 4.11, the attribute-groups introduce a new level in the XML tree. It is a one-to-one equivalent realization of the SITUATIONALSTATEMENTS model as defined in chapter 4.4. The two following syntactic variations represent the SITUATIONALSTATEMENT attributes as XML attributes. This has the advantage of compactness, but the disadvantage of restricted expressivity. The Mix version is a mixture


```

<statement>
  <subject>    a1  </subject>
  <auxiliary>  a2  </auxiliary>
  <predicate>  a3  </predicate>
  <range>      a4  </range>
  <object>     a5  </object>

  <start>      a6  </start>
  <end>        a7  </end>
  <durability> a8  </durability>
  <location>   a9  </location>
  <position>   a10 </position>

  <source>     a11 </source>
  <creator>    a12 </creator>
  <method>     a13 </method>
  <evidence>   a14 </evidence>
  <confidence> a15 </confidence>

  <key>        a16 </key>
  <owner>      a17 </owner>
  <access>     a18 </access>
  <purpose>    a19 </purpose>
  <retention>  a20 </retention>

  <id>         a21 </id>
  <unique>     a22 </unique>
  <replaces>   a23 </replaces>
  <group>      a24 </group>
  <notes>      a25 </notes>
</statement>

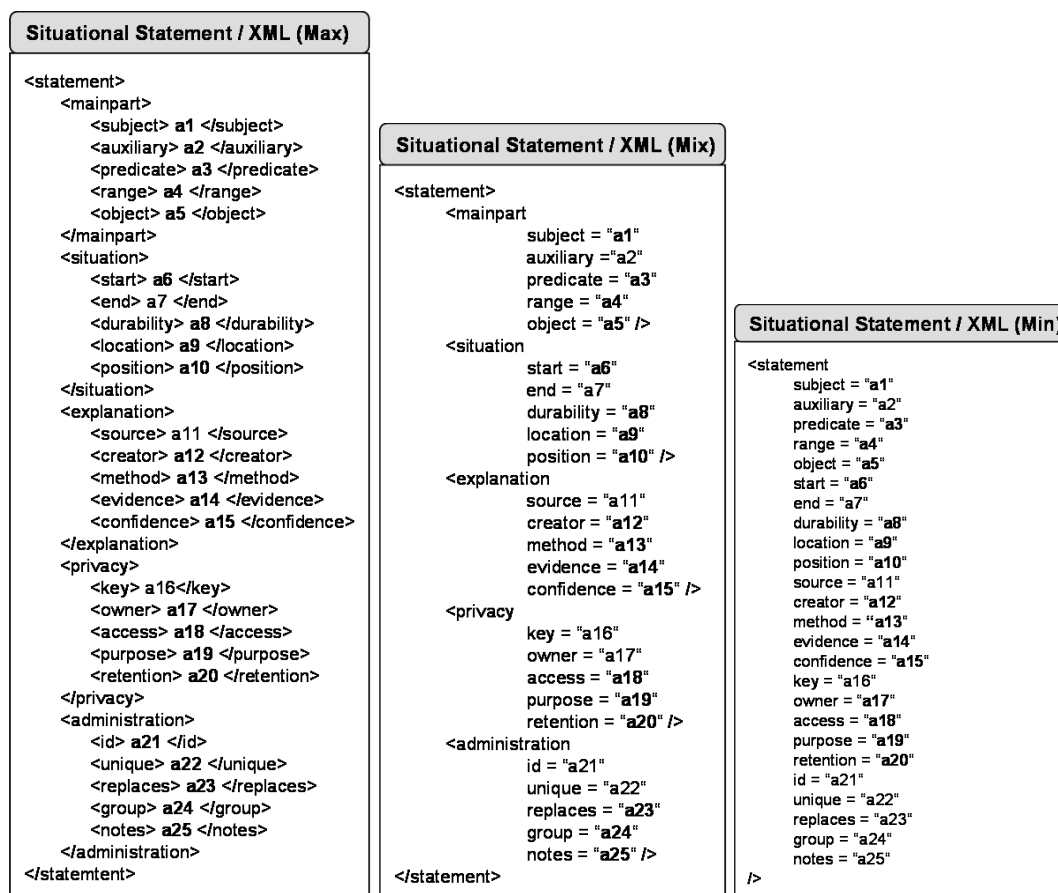
```

Figure 4.10: *SituationML/XML is the default representation*

between Max and Min and it turned out that statements in this version are especially good for being read by humans with ordinary browsers.

The minimal syntactic variation, on the right hand side in figure 4.11, only uses XML attributes and thus has the disadvantage of restricted expressivity within the variable. However, it is the premium choice if bandwidth for mobile systems is a problem and every single character counts.

The tools that were developed for handling SituationML can handle and transform all these variations. Thus, the individual application can choose which representation suits best. Of course, all XML applications can be used with namespaces. The recommended namespace is `st:` or `s:` and should be read as “statement”. It is mapped to <http://www.u2m.org/2003/02/situation/>. An example is given in appendix A.1.2 on page 206.

Figure 4.11: Three syntax variations for *SituationML*

4.4.3 SituationML/RDF and SituationML/OWL

Since most of the attribute values will be used like resources in RDF, the connection between *SituationML* and RDF is analyzed in this subsection. Because of its rich vocabulary RDF allows several possibilities of modeling the same information. Interesting for statements is the reification method, since the meta tags enrich the main sentence directly. Figure 4.12 shows *SituationalStatement/RDF_Graph(Reification)*, while the reified mainpart is simplified and does not incorporate the auxiliary-predicate-range concept yet.

The translation of this *RDF_Graph* model into *RDF_XML* is not presented in the appendix but it could easily be constructed. For our purposes the reification concept is not necessarily needed in its original RDF format if we move the focus from the mainpart to the whole statement itself, where the mainpart attributes are also used as pointers to the central identification point of the statement as shown in figure 4.13.

Representing *SITUATIONALSTATEMENTS* in this simple graph format leads to the

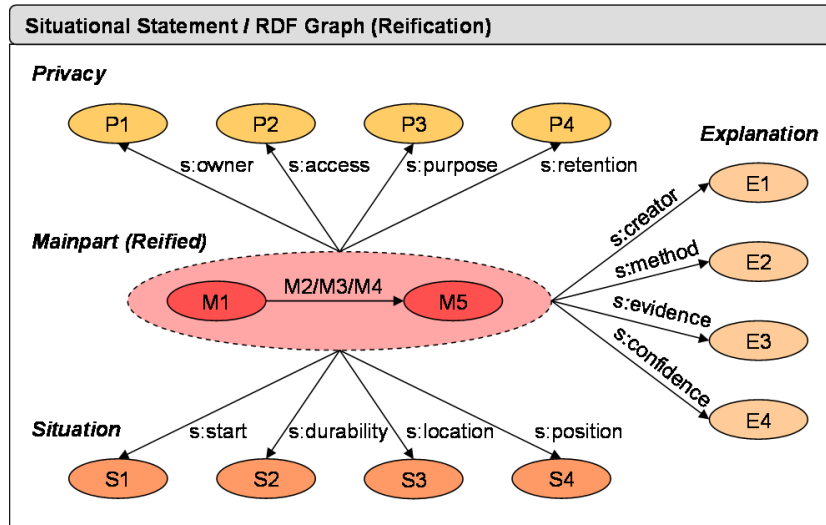


Figure 4.12: A SITUATIONALSTATEMENT schema in RDF graph notation in the variation with reification. The reified mainpart is enriched with some situation attributes, explanation attributes and privacy attributes. Each oval represents one resource. The attributes are realized as predefined RDF predicates, while their values can be defined as RDF objects

RDF_XML document as presented in figure 4.14. The similarity to the basic XML representation as shown in 4.10 on page 69 is remarkable. Because of the external definition of the semantic roles of the attributes and the implicit layered approach, no further XML structure is needed. With the help of SituationML/RDF_XML, the statements can straightforwardly be integrated into RDF ontologies for inferences. However, since most modern ontologies are represented either in the semantic web languages DAML+OIL or OWL, we have to allow SituationML to be integrated into these ontologies.

The major advantage of using the statement's identification as the RDF's subject and preferring the RDF graph without reification is that it is possible to choose the data type for every attribute, that is to say to choose between *URI reference* and *literal*, for the st:subject, the st:predicate and the st:object. As described in [Lassila and Swick, 1999], an RDF triple contains three components, namely the subject (1), which is a URI reference (or a blank node), the predicate (2), which is a URI reference, or the object (3), which is a URI reference, a literal (or a blank node). This means that only the RDF objects can contain literals as values. This hidden RDF feature turned out to be a disadvantage for the modeling purposes in the U2M approach. In the following section we will introduce extended resource identifiers that need to be represented as literals. With the chosen RDF structure of figure 4.14, we can handle this need. A list of real-life examples can be found online at the UbiWorld USERMODELSERVICE¹³.

The content of SITUATIONALSTATEMENTS may be represented in any suitable language.

¹³ <http://www.u2m.org/UbiWorld/UserModelServer.php?format=UserRDF>

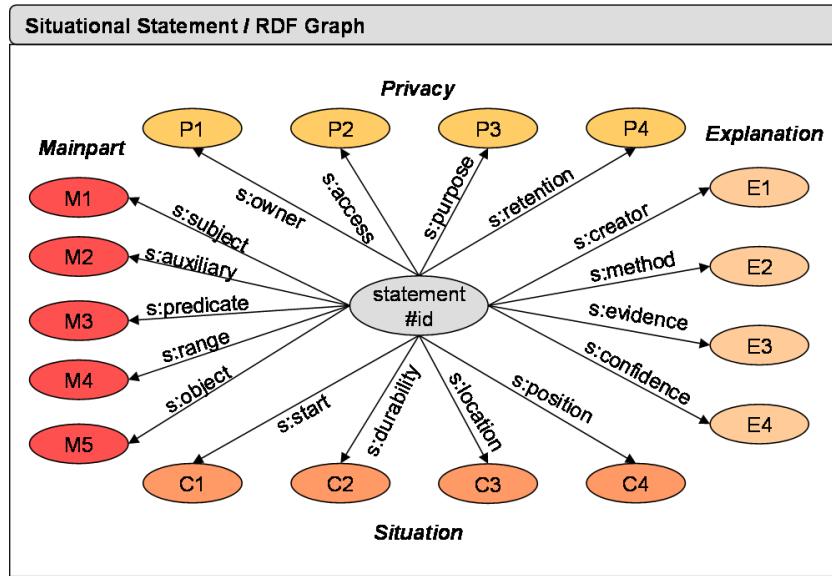


Figure 4.13: A SITUATIONALSTATEMENT schema in proper RDF/Graph without reification. The meta level attributes are arranged around the statement's identification

However, RDF seems to be an ideal candidate for knowledge representation if inference purposes are also important. The newly developed SituationML/RDF application is closely related to the *Dublin Core Metadata Initiative* (as described in section 3.2.1 on page 37), and the *Composite Capabilities/Preference Profile Specification* (as described in section 3.2.4 on page 41). SituationML/RDF is like Dublin Core about meta data, but the focus is different. It is not documents in a network that should be annotated but partial situations. Some Dublin Core elements are so general that they can also be used to describe situations. However, since the intended meaning and usage could be different, the attributes are introduced independently. Furthermore they are structured into groups and added with algorithmic entailments.

4.4.4 Discussion: Why did we introduce SituationML?

The question that arises is: why did we introduce SituationML to represent situational information rather than using description logics, first-order logics or situation semantics? Almost every knowledge representation language is able to represent our intended situational information. However, in [Larkin and Simon, 1987] it is argued, that even when two representations are informationally equivalent, which means that they carry the same information, their computational efficiency can vary enormously. The efficiency depends on the information-processing operators that act on them, which in turn depends on the tasks that should be performed with the information. SITUATIONALSTATEMENTS identify the following three main tasks: 1) *human readability*, 2) *situated retrieval* and 3) *inferences on meta-*

```

<rdf:RDF
  xmlns:rdf=http://www.w3.org/1999/02/22-rdf-syntax#
  xmlns:st=http://www.u2m.org/2003/situation#
  xml:base=http://www.u2m.org/statements>
<rdf:Description rdf:ID=a22>
  <st:subject> a1 </st:subject>
  <st:auxiliary> a2 </st:auxiliary>
  <st:predicate> a3 </st:predicate>
  <st:range> a4 </st:range>
  <st:object> a5 </st:object>
  <st:start> a6 </st:start>
  <st:end> a7 </st:end>
  <st:durability> a8 </st:durability>
  <st:location> a9 </st:location>
  <st:position> a10 </st:position>
  <st:source> a11 </st:source>
  <st:creator> a12 </st:creator>
  <st:method> a13 </st:method>
  <st:evidence> a14 </st:evidence>
  <st:confidence> a15 </st:confidence>
  <st:key> a16 </st:key>
  <st:owner> a17 </st:owner>
  <st:access> a18 </st:access>
  <st:purpose> a19 </st:purpose>
  <st:retention> a20 </st:retention>
  <st:id> a21 </st:id>
  <st:unique> a22 </st:unique>
  <st:replaces> a23 </st:replaces>
  <st:group> a24 </st:group>
  <st:notes> a25 </st:notes>
</rdf:Description>
</rdf:RDF>

```

Figure 4.14: *SituationML/RDF_XML defines each statement as an `rdf:Description`. The variables `a1` to `a25` serve as placeholders for the actual content*

level information. Representations in logics are more *sentential* like the propositions in a text than *diagrammatic*. However, SituationML is more *diagrammatic* since it preserves with its layered approach explicitly the relation between the information about the meta data and the mainpart of the statements, which are important for the efficiency in the main tasks number 2 and 3. In [Larkin and Simon, 1987] it is also argued that human abilities to recognize information are highly sensitive to the exact form and representation in which the information is presented to the senses. Ease of recognition may be strongly affected by what information is explicit in a representation, and what is only implicit. We argue, that it is an advantage for the human readability if the meta-level information is represented together with the mainpart content itself and not distributed in sentential form over several propositions. Furthermore the representation of the semantic web resource locators within SituationML (the pointers to the ontology) has been simplified as demonstrated in the following section about the extended resource identification.

SituationML shares several ideas with the theory of situation semantics. The

introduced concept of `SITUATIONALSTATEMENTS` forms a direct extension to the concept of infons, see section 2.2.3. However, the original framework of situation semantics is not used directly in our approach, since roles like ownership and privacy for example are not defined there. Furthermore, there exists no XML version or semantic web representation of situation semantics, which was one of our main design constraints.

The following section deals with the remaining question of the possible values and their representation of the variables from `a1` to `a25` as introduced in figure 4.10 in section 4.4 on page 69, or seen in figure 4.14.

4.5 Extended Resource Identification

This section looks at the issue of resource identification in RDF and extends the concepts of URIs especially for `SituationML`. *Mary* or *Peter* could for example be the subject of a user model statement. For an isolated, experimental blocks-world system `<subject>Mary</subject>` would be sufficient to represent the person Mary as pure literal. However, the question that arises is: *how can entities be referenced uniquely and efficiently in distributed real-world applications?* This question is especially important for the research task of `UBIQUITOUS USER MODELING FOR SITUATED INTERACTION` since globally unique identifiers are needed. Furthermore, these globally unique identifiers should still be human readable and secondly, short versions of these references should exist for efficiency reasons.

There are two approaches to introduce globally unique identifiers. The first one is the so-called GUID approach where more than 99,999% uniqueness is granted through the generation of a large random number that is calculated on the basis of the actual time in milliseconds together with a directed encryption method. The second one is the approach of uniform resource identifiers (URI) that was introduced in the internet, especially in the semantic web, see [Berners-Lee et al., 1998], [Lassila and Swick, 1999], [Champin, 2001] or section 2.3.1 for a detailed introduction. Both approaches to globally unique identifiers are used in `SituationML`: the statements receive GUIDs according to their time of appearance, while the ontological classes, their instances and the entities in the real-world are mapped to extended versions of URI references.

Finally, in the last subsection, the combination of unique identifiers into structured expressions is introduced. This so-called *UbiExpression* allows for statements about groups like *Mary and Peter*, for ontological entailments like class-extensions, as well as for abbreviations of long URIs. The interesting point about these new *UbiExpressions* is that they fulfill the requirements of proper RDF syntax because of our design decisions in chapter 4.4 where all `SituationML` attributes can be filled with RDF literals.

4.5.1 Extended URI References with Namespaces

SituationML is - like RDF - about describing resources. It can identify resources with qualified uniform resource identifiers. “Qualified URIs” are URIs with the slight difference that they can have an optional fragment identifier: a text added to the URI with a “#” between them. Resources in this sense can be considered as being the basic entities of a graph or conceptual mappings to entities. SituationML considers every qualified URI (with or without fragment identifier) as a full resource by itself. Since every qualified URI is unique, one could consider resources as linking unique identifiers to the referred objects and concepts. Figure 4.15 shows an example of two qualified URIs. The idea behind the qualification is that they allow parts of documents to be referenced as resources. The document `UserModelOntology` for example defines, among many others, the concept of *Cognitive Load*, (see chapter 5 for the ontology), and now this concept can be used as a resource, even if it is not a document by itself.

```
(1) http://www.u2m.org/UbisWorld#Peter
(2) http://www.u2m.org/UserModelOntology#CognitiveLoad
```

Figure 4.15: Two qualified uniform resource identifiers, (1) for a person in the context of *UbisWorld* with the name *Peter*, and (2) for the user modeling concept of *cognitive load*

Although resources are not in the general focus of attention, they need to be analyzed in detail in order to keep the representation simple. Qualified URIs can be generally abbreviated by XML namespaces. However, this namespace abbreviation is only possible in the RDF tags but not within the RDF content elements. Thus the two qualified uniform resource identifiers with namespaces in figure 4.16 cannot be used in SituationML/RDF.

```
(1) u2m:UbisWorld#Peter
(2) u2m:UserModelOntology#CognitiveLoad
```

Figure 4.16: Two qualified uniform resource identifiers with the namespace *u2m* :

4.5.2 UbisIdentifier (Ubid)

For the sake of clarity, one would prefer to use non-expanded notation like the ontology prefixes `Gumo:` and `UbisWorld:`, instead of `http://www.u2m.org/Gumo#` and `http://www.u2m.org/UbisWorld#` respectively, as shown in figure 4.17.

```
(1) UbisWorld:Peter
(2) Gumo:CognitiveLoad
```

Figure 4.17: Two resource identifiers with the prefixes *UbisWorld:* and *Gumo:*

These resource identifiers are pure literals and no longer URIs. They are not yet standardized but they are well formed for human readers (in comparison to figure 4.15), they

contain the same information as the full URI references and could thus be preprocessed into ordinary URIs. The idea of namespaces could be used within RDF content elements with standardized preprocessing.

A second problem that occurs in real-world naming is the fact that many entities have the same name. Several people could have the same first name or family name. Thus, information for disambiguation has to be added to the identifier, independently from the namespace. In large emailing systems, a counter is often added as a suffix to the identifying name. A second example is, if an ontology defines the two concepts of the animal *mouse*, and the computing device *mouse* in one document, one has to distinguish between these two meanings. In the WordNet¹⁴ research approach, see section 3.3.3 for a description, disambiguation is also realized by adding a counter to the semantical concept, but only to those concept names that show different meanings or semantical readings.

Bearing in mind the need for human readability and machine-processable names, we introduced the following new concept: We add a six digit number¹⁵ to every concept and entity name, together with a separating dot between the identifier's name and the identifier's number. With this syntactical convention, we can mostly omit the namespaces but it is still possible to distinguish between simple literals in parallel, and cover the problem of disambiguation-numbering in one step. Figure 4.18 shows two, so-called *UbisIdentifiers*.

- (1) `Peter.210004`
- (2) `CognitiveLoad.340050`

Figure 4.18: Two *UbisIdentifiers* with a separating dot and exactly six digits

These identifiers can omit the ontology denoting namespace because they refer per default to the *UbisWorldOntology*, which is defined in chapter 5. External ontologies can of course also be used, however mostly not with *UbisIdentifiers*, but with ordinary uniform resource identifiers. If external ontologies also use the same naming convention as *UbisIdentifiers*, together with the same ranges of numbers, then the concept of namespaces had to be applied. *UbisIdentifiers* can be seen as abbreviated references to ordinary URIs. After the preprocessing step, `Peter.210004` will be mapped to `http://www.u2m.org/UbisWorld#Peter`, or if the number is needed for disambiguation, to `http://www.u2m.org/UbisWorld#Peter.210004`.

Figure 4.19 shows seven different possibilities to denote a reference to the person Peter. All seven versions¹⁶ have their advantages and disadvantages and will be used for different purposes.

¹⁴WordNet homepage: <http://www.cogsci.princeton.edu/~wn/>

¹⁵Restricting *UbisIdentifiers* to six digits allows the ontology to carry up to 1 million concepts, which seemed to be sufficient for prototyping aspects. A debatable advantage of six-digit-numbers is that they can be read out and remembered easily as two three-digit-numbers: for example 204109 could be read as: *two-hundred-four, one-hundred-nine*

¹⁶The second reference will be defined in the EBNF grammar in figure 4.20

- (1) Peter
- (2) ubis(Peter)
- (3) Peter.210004
- (4) UbisWorld:Peter
- (5) <http://www.u2m.org/UbisWorld#Peter>
- (6) <http://www.u2m.org/UbisWorld#Peter.210004>
- (7) [map\(http://www.anydomain.org/anyOntology#Pepe\)](http://www.anydomain.org/anyOntology#Pepe)

Figure 4.19: Seven Possible identifiers to refer to the entity “Peter”: (1) purely literal, not pointing to an ontology, (2) abbreviated UbiIdentifier without the UbiId part, possibly pointing to several entities, (3) unique UbiIdentifier, (4) namespace-abbreviated URI, (5) UbiWorld URI, (6) UbiWorld URI with disambiguating UbiId, (7) mapping from an external ontology

4.5.3 UbiExpression (Ubex)

Now, the combination and grouping of identifiers is analyzed. Instead of representing only one resource reference per attribute, we allow a whole list of resource identifiers. If *Mary and Peter play together*, for example, both should form the subject of one statement `<subject> Mary.210005 | Peter.210004 </subject>`, rather than using two statements. This contributes to the actual research issue of group modeling additionally to single user modeling. Alternatively, one could introduce and identify the group of Mary and Peter as one referenceable unit, or as done in RDF, introduce an unspecified “blank node” that connects Mary and Peter to one group without introducing a new concept. However, it turned out that the introduction of the list structure on the level of attribute values and the level of identifiers forms an expressive element of syntactic sugar. Instead of using commas ‘,’ to indicate a new list element, vertical lines ‘|’ are used as separators.

Once we have implemented preprocessor functionality into the basic syntactic structure of SITUATIONALSTATEMENTS and introduced the list structure, we can go further and bring - for example - ontological inferences down to the level of statements. Instead of naming all instances (A-Box) like *Mary* and *Peter* that correspond to a concept (T-Box) like *student*, we could wish to write `<subject> extension( concept ) </subject>` and simulate “for-all statements”. This small, but fundamental step integrates the semantic web ontology with the syntactical representation level, not only by referencing with pure URIs like RDF does, but with the ontological extension functionality. Other ontology functions like `<subject> ancestors( concept ) </subject>`, have been implemented.

A third request for the design of UbiExpressions came from the research area of ontology merging. Since the SituationML approach allows any ontology to be referred to, we have to deal with the fact that several ontologies are used simultaneously by different systems, which leads to same concepts with different names on the one hand and same instances or individuals with different names on the other hand. At this syntactical level of

identifier-preprocessing, a mapping function fits perfectly into the evaluation of `UbisExpressions`: `<subject> map( Identifier ) </subject>`

Figure 4.20 shows the grammar that defines `UbisExpressions` in extended Backus Naur format in the version of [Scowen, 1998]. Nonterminals are not marked but terminals are written between two apostrophes. Underspecified nonterminals are written between quotation marks.

```

UbisExpression  -->  UbisExpression '|' UbisExpression
                  |   UbisFunction
                  |   UbisElement

UbisFunction    -->  'extension(' UbisIdentifier ')'
                  |   'ancestors(' UbisIdentifier ')'
                  |   'ubis(' UbisName ')'
                  |   'map(' UbisElement ')'

UbisList        -->  UbisList '|' UbisList
                  |   UbisElement

UbisElement     -->  UbisIdentifier
                  |   "RDF URI reference without '|' "
                  |   "RDF Literal without '|' "

UbisIdentifier  -->  UbisName '.' UbisId
                  |   UbisId

UbisName        -->  "String not starting with a digit and without '|' "

UbisId          -->  "Number with exactly 6 digits"

```

Figure 4.20: The EBNF grammar of `UbisExpression` that defines the vocabulary and strings that can be interpreted or used as values for statement attributes

The two new concepts that were not mentioned so far in this section are `UbisElement` and the `ubis( UbisName )` concept. The first one states that apart from `UbisIdentifiers`, ordinary RDF URI references and RDF literals without the special character `|` are also `UbisExpressions`. Thus `SituationML/RDF` is a superset of RDF “modulo `|`”. However, since statements with the list constructor `|` can be transformed into several statements without lists, every `SituationML/RDF` document can be translated into an ordinary RDF document. The second concept states that the transformation of a `UbisName` - without the `UbisId` - into one `UbisIdentifier` or into a list of `UbisIdentifiers` is offered as another abbreviation service. This functionality is important in order to distinguish between abbreviated `UbisIdentifiers` and general RDF literals.

To summarize, *UbisExpression* is an extension of RDF node elements that makes it possible to abbreviate URIs into smaller and more readable `UbisIdentifiers`, which represent an entity within the `UbisWorld` ontology. Technically speaking, a preprocessor maps all

kinds of abbreviations into lists of URIs and literals. Furthermore, by this extension the gap between ontological reasoning and pure knowledge representation has been filled with the help of *UbiFunctions*. Thus the variables from `a1` to `a25` as introduced in figure 4.10 in section 4.4 can carry complex preprocessable - but still ordinary - RDF literals or URIs as values.

To conclude the section about the syntax analysis of `SITUATIONALSTATEMENTS` and `SituationML`, an example statement with `UbiIdentifiers` is presented in figure 4.21.

```
<rdf:Description rdf:ID="statement1128">
  <st:subject>FinePixA202.230060</st:subject>
  <st:auxiliary>HasProperty.600100</st:auxiliary>
  <st:predicate>Price.820035</st:predicate>
  <st:range>Text.640002</st:range>
  <st:object>199 Euro</st:object>
  <st:start>2004-01-01T00:00:00</st:start>
  <st:durability>Month.520050</st:durability>
  <st:creator>Anonym.210020</st:creator>
  <st:method>ManualChangeWithEditor.910040</st:method>
  <st:confidence>0.99</st:confidence>
  <st:access>Public.640121</st:access>
  <st:purpose>Commercial.640131|Research.640132</st:purpose>
  <st:retention>Month.640142</st:retention>
</rdf:Description>
```

Figure 4.21: Example statement in *SituationML/RDF* with *UbiIdentifiers* and literals with the intended meaning that the camera *FinePix A202* has the actual price of 199 Euro.

Most attributes in this example statement carry `UbiIdentifiers` in their unabbreviated version as values, while the `<st:object>` and the `<st:start>` carry literals as values. The `<st:purpose>` attribute contains a list of two values as a `UbiExpression`.

4.5.4 UbiList (Ubli)

A `UbiEpression` without any `UbiFunction` is called a `UbiElementList`, or short `UbiList`. Every `UbiExpression` can be mapped to a `UbiList` that consists of a `|`-separated (read: pipe-separated) list of `UbiIdentifiers`, URI references or RDF literals respectively. Figure 4.20 contains the corresponding part of the EBNF grammar.

Compared to RDF node values, `UbiList` has the advantage of abbreviating blank nodes with multi child nodes and it allows abbreviated identifiers in addition to the uniform resource identifiers without namespace abbreviation.

